Project NOVA Official Technology Stack

Field	Value
Document ID	NOVA-TECH-001
Version	1.0
Status	`APPROVED`
Owner	Lead Software Engineer
Dependencies	NOVA-SPEC-001, NOVA-SPEC-003
Revision History	v1.0 Initial Stack Approval

---

1. Purpose and Scope

This document provides the exhaustive, legally binding technology decisions for Project NOVA. To prevent third-party bloat, every technology explicitly selected here has been justified. The introduction of any new external dependency requires an Architecture Decision Record (ADR) and a revision of this document.

---

2. Core Language & Toolchain

Python 3.12+

- **Purpose:** Primary programming language for the Kernel and Platform.
- **Justification:** The undisputed standard for AI/ML ecosystems. Python 3.12 introduces substantial performance improvements and advanced `asyncio` TaskGroups necessary for our Event Bus.
- **Alternatives:** Rust (Too slow to iterate on AI features), Go (Lacks native AI SDKs).
- **Trade-offs:** Global Interpreter Lock (GIL) limits true multi-threading, forcing us to rely on heavy asynchronous architectures.

uv

- **Purpose:** Package installer and virtual environment manager.
- **Justification:** Written in Rust, it is 10-100x faster than standard `pip`. Enforces strict resolution without the bloat of Poetry.
- **Alternatives:** Poetry, standard `pip`, Conda.
- **Trade-offs:** Relatively new tool; edge cases in Windows compilation might occasionally surface.

pyproject.toml

- **Purpose:** Centralized project configuration.
- **Justification:** Replaces `setup.py`, `requirements.txt`, and multiple config files with a single, modern standard PEP 518 manifest.

Ruff

- **Purpose:** Linter and formatter.
- **Justification:** Rust-based, replacing `flake8`, `isort`, and `black` entirely in a single sub-millisecond execution step.

Pyright

- **Purpose:** Static Type Checker.
- **Justification:** Extremely fast and strictly enforces Python type-hinting, which is mandatory for our heavily abstracted DI container interfaces.
- **Alternatives:** Mypy (Slower, harder to configure for complex generics).

pytest

- **Purpose:** Unit and Integration testing framework.
- **Justification:** Industry standard. We will heavily utilize `pytest-asyncio` to test our asynchronous Kernel loops.

---

3. Kernel Infrastructure

asyncio (Native)

- **Purpose:** Concurrency, Event Bus, and System Loop management.
- **Justification:** Utilizing Python's built-in `asyncio.Queue` and `asyncio.TaskGroup` allows us to build a lightning-fast Publisher/Subscriber Event Bus without introducing external bloat like Redis or RabbitMQ for local communication.

In-House DI (ServiceLocator)

- **Purpose:** Dependency Injection.
- **Justification:** Originally considered `dependency-injector`, but opted to build a native, lightweight `ServiceLocator` to maintain absolute control over the Boot sequence and enforce the "No Bloat" constraint of Milestone 1.

structlog / Native logging

- **Purpose:** Structured JSON event logging.
- **Justification:** Machine-readable logs are required for the AI Reflection Engine to analyze system crashes and trace asynchronous event flows.

Pydantic & Pydantic Settings

- **Purpose:** Schema validation and environment configuration.
- **Justification:** Guarantees that capabilities define strict JSON schemas for their inputs/outputs, and handles `.env` secrets securely.

---

4. Data & Memory Providers

SQLite

- **Purpose:** Semantic memory, relational metadata, and task logging.
- **Justification:** Serverless, local, and zero-configuration. Perfectly fits the local-first ethos of NOVA.

SQLModel

- **Purpose:** ORM (Object-Relational Mapping).
- **Justification:** Combines Pydantic and SQLAlchemy into a single, type-safe API, heavily reducing boilerplate code.

ChromaDB

- **Purpose:** Local Vector Database for embeddings.
- **Justification:** Runs natively in-memory or on local disk via Python without requiring a Docker container. Essential for RAG (Retrieval-Augmented Generation) memory.
- **Alternatives:** Qdrant (Heavier), FAISS (Harder to manage metadata).

---

5. OS Capability Providers

Playwright

- **Purpose:** Browser Automation.
- **Justification:** Modern, async-first, and highly resilient compared to Selenium. Can extract the DOM Accessibility Tree seamlessly for AI ingestion.

PaddleOCR

- **Purpose:** Screen text extraction.
- **Justification:** Extremely accurate for UI text and runs entirely locally, preventing sensitive screenshots from being sent to external APIs.
- **Alternatives:** Tesseract (Lower accuracy), EasyOCR.

OpenCV & mss

- **Purpose:** Screen capture and bounding box overlay processing.
- **Justification:** `mss` provides ultra-fast native screenshots, while OpenCV handles the heavy image matrix manipulations required by the Vision capability.

faster-whisper

- **Purpose:** Speech-to-Text (STT).
- **Justification:** Local execution of the Whisper model on CPU/GPU. Highly accurate and prevents audio streaming to external clouds.

Piper

- **Purpose:** Text-to-Speech (TTS).
- **Justification:** Extremely fast local TTS optimized for low latency, crucial for real-time conversational interfaces.

---

6. Presentation Layer & Deployment

PySide6 (Qt)

- **Purpose:** Desktop GUI (System Tray and Overlays).
- **Justification:** Official Python bindings for Qt. Allows us to create transparent, click-through overlays on Windows to highlight UI elements the AI is "looking" at.
- **Alternatives:** Tkinter (Ugly), PyQt6 (GPL licensing issues).

PyInstaller

- **Purpose:** Application Packaging.
- **Justification:** Bundles the Python runtime, SQLite, and ML models into a single executable for distribution.

---

7. Acceptance Criteria

- The stack documented here perfectly matches the dependencies installed in `pyproject.toml`.
- Any architectural deviations from this document trigger an official Architecture Decision Record (ADR) review.